

CLINICAL TRIAL INTELLIGENCE ON EXASOL — WHAT WE BUILT

Demo 1 • Pharma for Open Source India 2026. Every number in this document was queried from the running system, not quoted from a design note. Where a figure is an estimate or a lower bound, it says so.

AT A GLANCE

The question it answers	Who else is competing for my Phase 3 lung-cancer patients?
Why that is hard	Half the question is a <code>WHERE</code> clause; the other half exists only as free text
The finding	Recall 0.675 → 0.858 by adding one column . No model was retrained.
Engine	Exasol Personal, single node, on a laptop
Sources	ClinicalTrials.gov (12,404 trials) · PubMed (3,796 papers)
Storage	Native Exasol tables + Iceberg on object storage, joined in one statement
Surfaces	A Streamlit page, seven CLI steps, and an agent over MCP

SECTION 1: WHAT THE DEMO ARGUES

A pharma team planning a Phase 3 study asks who else is recruiting the same patients. That single question splits four ways, and only one part is easy.

PART OF THE QUESTION	WHERE IT LIVES	ANSWERABLE?
"Phase 3, NSCLC, recruiting"	coded registry fields	Yes — a <code>WHERE</code> clause
"... that exclude prior anti-PD-1"	free text inside one eligibility blob	No column holds it
"What phase are these trials?"	<code>PHASE</code> — but 34.4% say <code>NOT_APPLICABLE</code>	Misleading if filtered naively
"Which of these ever published?"	not in the warehouse at all	Needs a second corpus

The claim. Putting both halves in one engine, and recovering one column the registry never had, beats what a better retrieval model could have achieved. Structure beats scale, when you can get structure.

SECTION 2: DATA SOURCES

SOURCE	WHAT WE TAKE	HOW IT JOINS	COMMITTED TO THE REPO?
ClinicalTrials.gov API v2	12,404 interventional oncology trials, 2015+, NSCLC and breast	—	Yes, 15 MB gzip
PubMed E-utilities	3,796 papers that cite one of those trials	NCT_ID , carried by PubMed as a DataBank accession	Yes, 2.7 MB gzip

Both snapshots are committed so a demo never depends on conference wifi.

On the join key. PubMed records the registry number as a structured accession, so trial → paper is an equi-join on NCT_ID . No title matching, no fuzzy logic, nothing that needs a caveat. This is the one place in the demo that needs no retrieval at all.

Sources considered and not used: EU CTR/CTIS, OpenAlex, Drugs@FDA, EMA EPARs. Each joins on a drug name or a second registry identifier that has to be reconciled — weaker than an NCT id, and a wrong link on screen is worse than no link.

SECTION 3: BUILD TIME — RUNS ONCE, OFFLINE

None of this happens during a demo. Three independent chains.

3A. Lane A — the registry becomes tables

```
ClinicalTrials.gov snapshot → ingest/shred.py → IMPORT FROM LOCAL CSV FILE
```

shred.py turns nested JSON into flat rows and, while doing so, **recovers the inclusion/exclusion split from the free-text headings** — the registry has no field for it. 268,912 criteria, one row per sentence.

OUTPUT	ROWS
CT.TRIALS	12,404
CT.ELIG_CHUNKS	268,912
CT.TRIAL_CONDITIONS , _COUNTRIES , _OUTCOMES , _ARMS , _INTERVENTIONS	supporting

Then sql/03_semantic_layer.sql builds the views: CT.V_LANDSCAPE (one row per trial), V_TRIALS , V_TRIAL_GEOGRAPHY , V_TRIAL_ENDPOINTS , V_TRIAL_DESIGN .

What the layer derives, and what it admits

DERIVED COLUMN	COVERAGE	STATED WHERE?
<code>CRITERION_SECTION</code> (INCLUSION / EXCLUSION / UNKNOWN)	97.3% resolved, 2.7% UNKNOWN	on screen
<code>PHASE_IS_STATED</code>	34.4% of trials (4,261) are NOT_APPLICABLE	on screen
<code>PRIMARY_ENDPOINT_CATEGORY</code> (18 categories from free text)	35.7% "Other / unclassified"	on screen
<code>REGION</code> (114 countries → 7 regions)	complete	on screen

3B. Lane B — the text becomes arithmetic

268,912 criteria → Docker (TF-IDF → SVD 96 dims → L2) → 25,815,552 rows in Exasol

Runs in Docker with **sklearn pinned to the version the Exasol language container ships** (1.7.2 / numpy 1.26.4) — otherwise the analyzer pickle will not unpickle inside the UDF at query time.

The design idea. Exasol has no vector type and no vector index, so each 96-dimension vector is stored **long and narrow — one row per dimension**. Vectors are L2-normalised at build time, so a dot product *is* cosine similarity:

```
SELECT v.NCT_ID, v.CHUNK_ID, SUM(v.VAL * q.VAL) AS SIM
FROM   CT.ELIG_VECTORS v
JOIN   QV q ON q.DIM = v.DIM
GROUP BY v.NCT_ID, v.CHUNK_ID
```

Similarity search becomes an ordinary join and aggregate — the operation the engine is built for. No index to tune, no recall cliff, exact rather than approximate.

Also produced: BM25 tables (`CHUNK_TOKENS` , `TERM_IDF` , `CHUNK_LEN`) and `elig_model.pkl` , which goes into BucketFS.

3C. Lane C — the literature stays where it is

PubMed snapshot → lake/load_iceberg.py (pyiceberg) → Iceberg tables on MinIO

TABLE	ROWS
<code>ct.publications</code>	3,796
<code>ct.trial_publications</code>	4,627 trial–paper links

Never loaded into Exasol. Two tables written as Parquet behind an Iceberg REST catalog. Exasol is never told about the files.

SECTION 4: WHERE THE DATA LIVES

TIER	WHAT	WHY THERE
Native Exasol	trials, criteria, 25.8M vector rows, BM25 tables, semantic views	the hot path — needs to be fast
BucketFS	<code>elig_model.pkl</code> (94 MB), lakehouse engine <code>.so</code> , Rust language container	read by UDFs at query time; on Exasol Personal this is an ordinary host directory
Iceberg on MinIO	publications, trial↔paper links	large, rarely changes, owned by another team — read in place

The lakehouse is reached through a **virtual schema** (`CT_LAKE`) backed by `exasol-labs/lakehouse-engine-rs` — DataFusion running inside a Rust UDF. The planner joins it to native tables like any other schema.

SECTION 5: QUERY TIME — THREE ROUTES, ONE ENGINE

Three genuinely different kinds of question take three different paths. All three return an answer citing NCT ids.

5A. Route 1 — an agent asks *(app tab 6)*

a question → Claude, over the Exasol MCP server → SQL the agent wrote → answer

- `exasol-mcp-server`, read-only. The agent lists and describes the schema, then runs its own query.
- No SQL is written for it.** Different statement every time.
- Same views, same grants. `CT_LAKE` is just another schema to it.
- Model: `claude-opus-5`, tools supplied as MCP tools via the Anthropic tool runner.

Note. Query execution is **off by default** in the MCP server — 22 metadata tools and no executor until `EXA_MCP_SETTINGS={"enable_read_query": true, ...}` is set, which yields 24 tools including `execute_exasol_query`.

5B. Route 2 — the app asks *(app tab 5)*

the same question → vocabulary match (no LLM) → one fixed statement
→ vector + BM25 → RRF fusion → answer

`bin/split.py` matches the question word by word against vocabulary read live from `CT.V_LANDSCAPE`, producing the structured predicate, the section filter, and the residue that becomes the search text. **Same question, same SQL, every time** — which is what makes it safe to run on stage.

The statement is `sql/05_hybrid_search.sql.tpl`. The structured filter is applied **before** scoring, not after.

This route does **not** read the lakehouse. The template contains no reference to `CT_LAKE`.

5C. Route 3 — an analytical question (app tab 3)

```
"which completed trials ever published?"
→ CT.V_LANDSCAPE LEFT JOIN CT_LAKE.TRIAL_PUBLICATIONS → a number
```

No retrieval involved. A native view joined to Iceberg in object storage, in one statement across two storage tiers. **This is the only route that reads the lake.**

SECTION 6: THE SURFACES

SURFACE	WHAT IT IS	HOW TO RUN
Streamlit page	the booth screen — 7 tabs, plus a Walkthrough mode of 6 full-screen steps	<code>./app/run.sh</code> → <code>:8503</code>
CLI steps	the build, narrated — each step prints the SQL before running it	<code>./01_snapshot.sh</code> ... <code>./07_lake.sh</code>
Preflight	15 checks, GO / NO-GO	<code>./00_preflight.sh</code>
Probe UI	the original analyst tool, stdlib only	<code>python3 bin/ui.py</code> → <code>:8899</code>

Streamlit tabs: 1 the problem · 2 the solution · 3 the value · 4 architecture · 5 the demo with evidence · 6 the agent · 7 what this will not claim.

SECTION 7: MEASURED RESULTS

7A. Retrieval

	RECALL@20	SECTION PURITY
Text alone	0.675	0.763
With the recovered column	0.858	1.000
— hybrid questions	0.713 → 0.925	0.769 → 1.000
— polarity questions	0.600 → 0.725	0.750 → 1.000

7B. The two failures, both real and both left in

Stopword deletion. `no` is an English stopword, so *"No prior treatment with anti-PD-1"* and *"Prior treatment with anti-PD-1"* tokenise **identically** — provable live with one `CT.QUERY_TERMS` call. Fixed by the `CRITERION_SECTION` column, not by a bigger model.

Antonymy. Ask for *"EGFR mutation positive"* and at **rank 13** sits *"EGFR mutation or ALK mutation was negative"* (cosine 0.9455) — in INCLUSION, the half we asked for. `negative` is not a stopword: the token survives, the meaning does not. **The section filter cannot help here.** Left unfixed and documented.

7C. The lakehouse

Completed Phase 3 trials with no linked publication	65.8% (287 of 436)
Cost of reaching the lake	+159 ms over a native query

7D. The agent

Four questions, measured end to end, 19–34 s each, 2–4 turns:

QUESTION	RESULT
Phase 3 NSCLC recruiting, top sponsors	174 trials; Merck 10, AZ 7 — self-corrected after guessing a status value that does not exist
Trials excluding prior anti-PD-1	1,124 trials; used <code>CRITERION_SECTION</code> unprompted
Completed Phase 3 with a publication	149 of 242 — joined <code>CT_LAKE</code>
Biggest Phase 3 breast trials	noticed the top two are screening studies, not treatment

When the lakehouse was unavailable it reported the infrastructure fault and **refused to produce a number**.

SECTION 8: WHAT THIS WILL NOT CLAIM

LIMIT	DETAIL
Not independent ground truth	Eval gold sets are SQL predicates over the same corpus. Reproducible, not an outside judgement.
Not a transformer	TF-IDF + 96-dim SVD, ~18% of variance. A transformer would retrieve better — and would be just as blind to the word "no" .
Not proof of non-publication	65.8% is a lower bound on linkage . A paper that never cites its NCT number is invisible to the join.
Not a CRO portfolio	The registry has no CRO field; <code>LeadSponsor</code> is the pharma company.
Not billion-scale	25.8M rows on a single-node VM. Linear in rows × dimensions.

SECTION 9: OPERATIONAL NOTES

The VM clock is the demo-killer. Every S3 request is signed with the Exasol VM's clock. When the host sleeps, that clock freezes while the hardware clock stays correct, and every lakehouse query then fails with `403 PermissionDenied / RequestTimeTooSkewed` — which reads exactly like bad credentials and is not. Observed four times in one day, with skews from 10 minutes to 7.6 hours. `lake/up.sh` and `00_preflight.sh` now resync it automatically; the Streamlit page offers a one-click fix.

FAILURE	LOOKS LIKE	ACTUALLY
VM clock drift	S3 403, "bad credentials"	host slept; resync with <code>hwclock -s</code>
MCP has no query tool	agent can only describe schemas	<code>enable_read_query</code> is off by default
Stale UI process	page looks fine until you type	an old server still holding the port
Out of API credits	<code>ExceptionGroup: unhandled errors in a TaskGroup</code>	unwrap the exception group

SECTION 10: HOW TO RUN IT

```
./lake/up.sh           # object storage + Iceberg catalog, 2 containers
./lake/install_engine.sh # lakehouse engine into Exasol (idempotent)
./02_load_exasol.sh    # shred and load – the snapshot ships with the repo
./03_semantic_layer.sh # views, and the CRO tables
./04_build_vectors.sh  # embed offline, load 25.8M rows, install the UDFs
./work/lakeenv/bin/python lake/load_iceberg.py
./00_preflight.sh      # GO / NO-GO, 15 checks
./app/run.sh           # the booth screen on :8503
```

The agent tab additionally needs `ANTHROPIC_API_KEY` in a gitignored `.env`.

APPENDIX: FILE MAP

PATH	WHAT IT IS
<code>ingest/fetch_snapshot.py</code> , <code>fetch_pubmed.py</code>	source fetchers (snapshots are committed)
<code>ingest/shred.py</code>	JSON → CSV, and the section recovery
<code>ml/build_vectors.py</code> , <code>Dockerfile</code> , <code>parquet_to_csv.py</code>	the offline embedding
<code>sql/01...06</code>	tables, retrieval tables, semantic layer, UDFs, CRO tables
<code>sql/05_hybrid_search.sql.tmpl</code>	the canonical retrieval statement
<code>lake/</code>	compose file, engine installer, Iceberg loader, up/down
<code>bin/split.py</code>	the deterministic question splitter
<code>bin/search.py</code> , <code>exasql.py</code>	template fill and the single SQL route
<code>app/app.py</code>	the Streamlit page
<code>app/agent.py</code>	the MCP agent
<code>app/architecture.py</code>	the architecture diagram, drawn from live numbers
<code>app/check_diagram.py</code>	8 geometry checks on that diagram
<code>eval/run_eval.py</code> , <code>questions.json</code> , <code>results.json</code>	the measured retrieval eval